LEARN BEING FORWARD

Programming — C# Programming

Building console & basic .NET apps

COMPLETE LEARNING GUIDE

What You'll Learn

Writing larger, real console applications moves beyond individual language features into practical project structure — how a .NET project is organized on disk, how to read and validate user input robustly, how to split code across multiple files and namespaces sensibly, how to read configuration and command-line arguments, basic file I/O, and the .NET CLI commands used to build, run, and test a project day to day. This sub-module ties together everything from the earlier C# sub-modules into the shape of an actual small application, closing out the C# module with the practical, project-level knowledge needed to go from individual language features to a working, runnable program a user can actually invoke from a terminal.

1. Project Structure: .csproj, Program.cs, and Dependencies

A .NET console project is defined by a .csproj file — an XML project file specifying the target framework (net8.0, for example), any NuGet package dependencies, and build settings — and by convention contains a Program.cs file with the application's entry point. dotnet new console creates this minimal structure automatically. Adding a third-party library is done with dotnet add package PackageName, which downloads it from NuGet and records it as a dependency in the .csproj file, after which its types become available via using statements exactly like built-in .NET types.

```csharp
// Typical minimal Program.cs (top-level statements style)
Console.WriteLine("Starting application...");

// Typical commands run from a terminal in the project folder:
// dotnet new console -o MyApp      (scaffolds a new console project)
// dotnet add package Newtonsoft.Json  (adds a NuGet dependency)
// dotnet build                      (compiles the project)
// dotnet run                        (builds, then runs it)
```

2. Console I/O and Parsing Input

Console.WriteLine/Write send output to the terminal; Console.ReadLine() reads one line of text typed by the user, always returning it as a string (or null if input was redirected and reached EOF) — numeric input must be explicitly parsed with int.Parse (throws FormatException on invalid text) or the safer int.TryParse (returns a bool success flag and avoids the exception entirely), exactly mirroring the Dictionary TryGetValue pattern from the Collections sub-module. Always validating user input with TryParse rather than assuming it will always be well-formed is the single most important console-app input habit to build early.

```csharp
Console.Write("Enter your age: ");
string? input = Console.ReadLine();

if (int.TryParse(input, out int age)) {
    Console.WriteLine($"You are {age} years old.");
} else {
    Console.WriteLine("That wasn't a valid number.");
}
```

3. Organizing Code: Multiple Classes and Namespaces

As a console application grows beyond a trivial script, splitting it into multiple files — one class per file by convention, grouped into folders that map to namespaces reflecting the project's logical structure (MyApp.Models, MyApp.Services) — keeps the codebase navigable. C# resolves types across files within the same project automatically as long as the correct using directive (or fully-qualified name) is present; there's no separate header-file declaration step the way C/C++ requires, since the compiler processes the whole project together.

```csharp
// File: Models/Student.cs
namespace MyApp.Models {
    public class Student {
        public string Name { get; set; } = "";
        public int Grade { get; set; }
    }
}

// File: Program.cs
using MyApp.Models;

var student = new Student { Name = "Asha", Grade = 90 };
Console.WriteLine($"{student.Name}: {student.Grade}");
```

4. Command-Line Arguments and Basic Configuration

A console app receives command-line arguments through the args parameter of Main (static void Main(string[] args)) or, in top-level-statements style, directly through an implicit args variable — each argument arrives as a string and, just like Console.ReadLine, must be explicitly parsed into a numeric type if arithmetic is needed. For anything beyond a couple of simple flags, structured configuration (appsettings.json read via the Microsoft.Extensions.Configuration package) is the standard, more scalable approach used in real .NET applications, though plain args parsing is entirely sufficient for small utilities and learning exercises.

```csharp
// Top-level statements: `args` is available implicitly
if (args.Length < 2) {
    Console.WriteLine("Usage: dotnet run <num1> <num2>");
    return;
}

if (int.TryParse(args[0], out int a) && int.TryParse(args[1], out int b)) {
    Console.WriteLine($"Sum = {a + b}");
} else {
    Console.WriteLine("Both arguments must be valid integers.");
}
// Run as: dotnet run 3 4   ->   Sum = 7
```

5. Basic File I/O with System.IO

The System.IO namespace's File class provides simple, static, synchronous methods for common whole-file operations — File.ReadAllText/File.WriteAllText for an entire file as one string, File.ReadAllLines/File.WriteAllLines for line-by-line text — which is usually sufficient for small console utilities without needing the lower-level StreamReader/StreamWriter classes that offer more control for very large files or streaming scenarios. File operations can throw (FileNotFoundException, UnauthorizedAccessException, IOException for a locked file), so wrapping them in try/catch or checking File.Exists first is standard practice, exactly mirroring the fopen-NULL-check discipline from C's file handling.

```csharp
string path = "notes.txt";

File.WriteAllLines(path, new[] { "Line 1", "Line 2", "Line 3" });

if (File.Exists(path)) {
    string[] lines = File.ReadAllLines(path);
    foreach (string line in lines) {
        Console.WriteLine($"Read: {line}");
    }
}
```

6. The .NET CLI: build, run, and test

dotnet build compiles the project without running it, useful for catching compile errors quickly or preparing a release build; dotnet run compiles (if needed) and immediately runs the resulting executable, the everyday command used while actively developing; dotnet test runs any test projects referencing a testing framework (xUnit, NUnit, or MSTest) in the solution. Understanding which command does what — and that dotnet run implicitly triggers a build first if the source has changed — avoids confusion when a change doesn't seem to take effect (usually because an old build was run instead of a fresh one, or a build error was silently left unaddressed).

- dotnet run always rebuilds first if source files changed since the last build — there's rarely a need to run dotnet build separately before it during normal development.
- dotnet test discovers and runs test projects; it requires a referenced test framework package (xUnit is the most common default for new .NET projects).

7. Debugging a Console Application

Breakpoints set in an IDE (Visual Studio, VS Code with the C# extension) pause execution at a specific line, letting you inspect variable values and step through code one statement at a time — far more effective for understanding a bug's actual cause than scattering Console.WriteLine calls throughout the code, though the latter remains a quick, zero-setup technique for simple cases. Unhandled exceptions in a console app print a full stack trace to the console and terminate the process, which — unlike a crash in C from undefined behavior — at least tells you exactly where and why the failure happened, a direct benefit of the CLR's managed exception model covered earlier in this module.

Common Pitfalls

- Calling int.Parse on raw Console.ReadLine() input without validating it first, crashing on any non-numeric input instead of handling it gracefully with int.TryParse.
- Forgetting that Console.ReadLine() can return null (e.g., when input is redirected and reaches EOF), and not guarding against it before using the result.
- Hardcoding absolute file paths instead of relative paths or Path.Combine, breaking the moment the project is run from a different machine or working directory.
- Assuming dotnet build is required before every dotnet run — it's implicitly triggered when needed, so the extra step is usually unnecessary friction.
- Not wrapping File I/O operations in try/catch (or checking File.Exists first), crashing on a missing file or a permissions error instead of failing gracefully.
- Cramming all application logic into a single Program.cs file as the project grows, instead of splitting it into focused classes and namespaces early.

Key Takeaways

- A .csproj file defines a project's target framework and dependencies; dotnet new/add/build/run/test are the everyday CLI commands for managing it.
- Always validate Console.ReadLine()/command-line argument input with TryParse rather than assuming it is well-formed.
- Split growing console applications into multiple classes and namespaces early rather than keeping everything in one Program.cs file.
- System.IO.File's static methods (ReadAllText, WriteAllLines, etc.) cover most simple whole-file read/write needs without extra setup.

- dotnet run implicitly rebuilds when source has changed, making a separate dotnet build step usually unnecessary during normal development.

Learn More

Microsoft Learn — Tutorial: Create a console application
https://learn.microsoft.com/en-us/dotnet/core/tutorials/with-visual-studio-code

Microsoft Learn — .NET CLI overview
https://learn.microsoft.com/en-us/dotnet/core/tools/